Model Checking Concurrent Programs with SPIN

RW796 Software Verification and Analysis

Gabriella Scott
25935453

September 2, 2026

Introduction

This report covers Promela models of two concurrent programs and the verification of their correctness properties using Spin. The first program is a first-come first-served (FCFS) process scheduler, one of three policies supported by a shared-memory process manager. A team of OpenMP threads dispatches processes from a common ready queue, moves them to a waiting queue when a resource is unavailable, and updates a shared resource table, with nested locks protecting these structures and new processes arriving while the threads are already running. The second program is a distributed Othello engine running a parallel alpha-beta search. An MPI master hands candidate moves out to worker processes as they report back, propagates a shared alpha bound between them, and reduces the search depth partway through a round when the turn clock runs down. It communicates over three channels: MPI collectives to start a round, point-to-point messages to distribute and collect work, and a socket to the referee.

Parts of each program were omitted, some behaviour was replaced with nondeterministic choices, and the remaining parameters were fixed to small values. These three moves do not affect the results in the same way, so it helps to separate them with the usual pair of terms. An abstraction is *sound* for a property if every counterexample it reports corresponds to a real execution of the implementation, meaning there are no false alarms. It is *complete* if every violation the implementation can commit also appears in the model, meaning there are no missed defects.

Replacing concrete behaviour with a nondeterministic choice adds executions rather than removing them, which is why Promela treats nondeterminism as the standard way to build an abstraction [4]. That direction keeps completeness but puts soundness at risk, because the model can now do things the implementation never would. Fixing the parameters to small values and leaving components out works the other way: it removes executions, so a counterexample that survives is still a real one, but a property that passes has only been shown to pass inside the bound. Each abstraction is justified where it is introduced, and Section 1.1 and Section 2.1 give the specific limits for each program.

The analysis found defects in both programs. For the scheduler, 6 of the 7 checks failed, with the counterexamples tracing back to two faults in `manager.c`: a resource request that does not check whether the requester already owns the resource, and a dispatch loop that accesses shared process state without holding a lock. For the Othello engine, 4 of the 5 checks failed, with 3 faults identified in `my_player.c`, namely an initialiser escaping from the minimax minimising branch as if it were a score, comparisons between scores calculated at different search depths, and two referee message types for which the master loop has no corresponding branch. The last fault did not occur in any recorded match, illustrating the value of model checking in finding behaviours that testing may not expose.

Promela models, properties and verification records: `https://github.com/Gabriella-Scott/PA_Assignments`
Implementation under analysis (Program 1 & 2): `https://git.cs.sun.ac.za/Computer-Science/rw314/2025/project/25935453-rw314-projects`

1 Program 1: FCFS Process Scheduler

1.1 The Promela Model

The model abstracts the FCFS path of `manager.c`: `schedule_fcfs` together with `execute_instr`, `request_resource`, `release_resource`, `enqueue_pcb`, `dequeue_pcb` and `terminate`. It consists of an

`init` process that seeds the ready queue, three inlines standing for `terminate`, `request_resource` and `release_resource`, and a `Worker` proctype instantiated once per scheduling thread. The other two policies are not modelled: all three sit on the same queues, locks and resource table, so including them would have multiplied the state space without reaching a different class of defect.

Four constants fix the size of the model, each the smallest value that still shows the behaviour of interest. `NUM_WORKERS` is 2, because a single thread cannot interleave with anything. `NUM_PROCS` is 3, because with 2 processes one running and one blocked leaves the ready queue empty, and a third is needed before a process can be dispatched while another is already blocked. `NUM_RESOURCES` is 2, the smallest number admitting a circular wait [1]. `MAX_INSTR` is 3, enough for a process to request a resource, release it and request again, so that a release can wake a blocked process. Identifiers start at 1 so that 0 can stand for `NULL` and `resource_owner[r] == 0` reads exactly as `resource->allocated == NULL` does. Only `MAX_INSTR` is wrapped in an `#ifndef` guard, since it is the one parameter varied between runs; supplying it with `spin -D` means every result in Section 1.3 comes from the same committed file. Table 1 sets out the correspondence between the implementation and the model.

Table 1: Mapping between the implementation and the model.

`manager.c`	**Model**	**Reason**
Thread in `schedule_fcfs`	`active [NUM_WORKERS] proctype Worker`	The parallel region runs identical threads over the same loop, so one proctype replicated twice covers every thread the scheduler creates.
`readyq`, `waitingq`	Buffered channels `ready_q`, `waiting_q`	FIFO order is the only behaviour of the linked lists FCFS depends on. `waiting_q` also carries the resource id, which the implementation reads from the instruction record.
`terminatedq`	`pcb_state[] == TERMINATED`	Nothing is ever dequeued from it, so membership carries no information the state field does not already hold.
`pcb_t.state`	`pcb_state[]`	Direct, indexed by process id instead of reached through a pointer.
`next_instruction`	`instr_count[]`	The instruction list becomes a bounded counter, with `next_instruction == NULL` represented by `instr_count == MAX_INSTR`.
`resource_t.allocated`	`resource_owner[]`	The list searched by `strcmp` becomes an array indexed by id, and `allocated == NULL` becomes owner 0.
`q_locks[0]`, `omp critical`, `i_lock`, `r_lock`	`atomic` blocks	Mutual exclusion is all the locks provide here, and `atomic` supplies it without modelling acquisition [2]. Only the regions the implementation actually protects are made atomic.
`terminate()`	`check_terminate` inline	The same test, evaluated over the state array rather than by walking the queues.

OpenMP nested locks were used to protect the shared structure in the implementation, which let a thread reacquire a lock it already holds [7]; `schedule_fcfs` takes `i_lock` and then calls `execute_instr`, which takes it again. The model represents no locks at all, only the `atomic` blocks standing for the regions they protect. That is sound provided the locks cannot deadlock against each other, which is checkable by inspection: every FCFS path acquires them in the same order, and `release_resource`, the only function holding two queue locks at once, always takes `q_locks[1]` before `q_locks[0]`. With a consistent aacquisition order no circular wait between locks is possible: a thread that holds one lock only ever waits for a lock later in the fixed order, so the wait-for relation cannot contain a cycle [1]. One caveat is that a blocking statement inside an `atomic` sequence causes Spin to drop atomicity rather than report an error [2], but the sends on `ready_q` and `waiting_q` can only block if a PCB is enqueued twice, which is what `no_dup_ready` rules out.

What the model leaves outside an `atomic` block matters more. The assignment `running_p->state =`

RUNNING, the loop guard while (running_p->state == RUNNING) and the test if (running_p->state == WAITING) all touch shared PCB state with no lock held, since i_lock is released as soon as execute_instr returns. Those statements sit outside any atomic block in the Worker loop, so pan is free to interleave the second worker there. Wrapping them in atomic for tidiness would have left the model unable to find the defect in Section 1.3.

Listing 1: Unprotected reads in schedule_fcfs, abridged

```
running_p->state = RUNNING; /* no lock held */
while (running_p->state == RUNNING) /* no lock held */
{
  ...
  omp_set_nest_lock(&i_lock);
  ...
  execute_instr(running_p);
  omp_unset_nest_lock(&i_lock); /* lock released here */

  load_new_processes();

  if (running_p->state == WAITING) /* no lock held */
  {
    break;
  }
  running_p->next_instruction = running_p->next_instruction->next;
  ...
}
```

Two arrays in the model do not have direct counterparts in manager.c; executing[] keeps track of how many workers are currently processing a PCB, while in_ready[] tracks how many times a PCB appears in the ready queue. Since neither is used in a guard, they cannot enable or block any transitions; only provide the state needed by pcb_mutex and no_dup_ready.

The instruction stream is where the model makes the biggest abstraction, because the implementation reads instructions from a linked list created by the loader, where each instruction has a type and resource name. The model does not include these instruction records; instead, select non-deterministically chooses a resource, while a two-branch if non-deterministically chooses between a request and a release [4]. Because pan explores both branches, the model covers every possible instruction sequence of up to MAX_INSTR steps, including sequences that the actual loader would not produce. Table 2 lists what was left out and what each omission costs.

Table 2: What the model leaves out, and what each omission costs.

Omitted	**Cost**
Process names, strcmp lookup	None → Resources are found by index (instead of by walking a list).
Logging	None → It writes to a file and touches no scheduling state.
omp barrier, omp single	None → They run after every process has terminated, so no property here can distinguish them.
The "resource does not exist" path	Small but real → The implementation reaches the same enqueue_pcb(..., WAITING) branch whether a resource is held by another process or absent entirely; the model represents only the first.
Dynamic arrivals and a_lock	Not free → load_new_processes can add to the ready queue while terminate() is counting it, so omitting arrivals removes a race the model cannot then find.
detect_deadlock	Not free → Deadlock is checked by Spin's invalid end state detection instead, which asks whether the system halted where it should not have rather than whether a circular wait exists.

A worker can try to dequeue a process before the ready queue has been filled, because the active proctypes start at the same time as init. The empty(ready_q) branch handles this case, but it also causes the simple liveness failures discussed in Section 1.3.

Limits on interpretation

The model was written by hand, so a counterexample is treated as a possible defect until its interleaving has been checked against the source code. This is done for each finding in Section 1.3. On the other hand, a property that holds only applies to the model configuration of 3 processes, 2 workers, 2 resources, and `MAX_INSTR` steps. One important assumption is that the model treats unprotected reads of `running_p->state` as ordinary reads that see the most recent value written. OpenMP does not guarantee this behaviour without a `flush` [7]. Consequently, the real program may have behaviours that are not represented in the model. This means that the identified defects are a lower bound on the problems with the unlocked dispatch loop.

1.2 Correctness Properties

Seven checks were run: 6 properties written as named `ltl` blocks in `model.pml`, plus a deadlock check that is not an LTL property. Table 3 gives each formula as it appears in the file, where `[]` is □ and `<>` is ◇ [5]. 4 are safety properties, which say a bad state is never reached and are violated by a finite prefix, and 2 are liveness properties, which say something good eventually happens and can only be violated by an infinite execution [5]. Promela has no quantifier, so every conjunction over the three process ids is written out by hand.

Table 3: The six LTL properties as written in `model.pml`.

Property	Class	Formula
`pcb_mutex`	safety	`[] (executing[p] <= 1)` for each `p`
`no_self_wait`	safety	`[] (waiting_for[p] == 0 || resource_owner[waiting_for[p]] != p)`
`waiting_consistent`	safety	`[] (pcb_state[p] != WAITING || waiting_for[p] != 0)`
`no_dup_ready`	safety	`[] (in_ready[p] <= 1)`
`all_terminate`	liveness	`<> (pcb_state[1] == TERMINATED && ... && pcb_state[3] == TERMINATED)`
`ready_dispatched`	liveness	`[] ((pcb_state[1] == READY) -> <> (pcb_state[1] == RUNNING))`

`pcb_mutex` is an important invariant because the PCB represents a unit of work, so only one thread should process it at a time. The dequeue operation is protected, but the code after it is not.

`no_self_wait` checks the behaviour of `request_resource`. The function only checks whether `resource->allocated == NULL` and does not check whether the resource is already owned by the requesting process.

`waiting_consistent` checks that a PCB in state `WAITING` is actually waiting for a resource, since the state and resource information are updated separately without a lock protecting both operations. It is considered together with `pcb_mutex`, because it shows how overlapping access can lead to incorrect state that affects later decisions.

`no_dup_ready` checks the three places that add processes to the ready queue: the long-term scheduler, `load_new_processes`, and `release_resource`. A duplicate entry could cause the same process to be dispatched twice and lead to another `pcb_mutex` violation that cannot be prevented just by locking the dispatch loop.

`all_terminate` is checked separately from the program's own stopping condition because they don't ask the same question. `terminate()` returns TRUE when all processes have been placed in either the waiting or terminated queues. This means a process that is blocked forever can still allow the workers to exit and the scheduler to report that it has finished.

`ready_dispatched` checks whether the system actually serves processes before stopping. It is only checked for process 1 because the 3 processes are symmetric. FCFS does not use a priority field, and all 3 instruction sequences are generated using the same nondeterministic choice. This is an assumption based on symmetry rather than a formal proof, and it is the one property in Program 1 that was weakened

to reduce the verification cost. Both liveness properties are checked under weak fairness, as discussed in Section 1.3.

Deadlock is checked with Spin's invalid end state detection, which asks whether any process is stopped somewhere other than a valid halting point [5].This matches the standard definition of deadlock as a state in which the system as a whole has halted while at least one component could still continue [1], and is broader than asking whether a circular wait exists; it catches a process blocked on a guard that can never become executable, whatever the reason. Since `pan` disables the check while a never claim is active, this run uses a separate claim-free verifier [3].

Three things were deliberately not checked, namely an assertion bounding `instr_count` was removed, because the deadlock trail already shows the same overrun and an active assertion would let other runs stop on it rather than on their named property. `detect_deadlock` is not verified, because it runs inside `#pragma omp single` and contains no concurrency. Lock reentrancy is not modelled, because no property depends on nesting depth, only on which accesses are protected.

1.3 Verification Results and Analysis

Each run selects a specific claim using `./pan -a -N name`. The deadlock check uses a second binary compiled from the same generated `pan.c` with `-DNOCLAIM`. This removes the claims from the verifier, rather than from the model itself [3]. The value of `MAX_INSTR` is also supplied to `spin` on the command line, ensuring that all results in Table 4 use the same source file.

Two extra flags are used on the larger runs. `-m` increases the maximum search depth from the default of 10,000, while `-w26` increases the hash table size to 2^{26} entries. The larger hash table was needed because the default 2^{19} entries caused many hash conflicts at this scale [3].

Table 4: Verification results. Depths and state counts as reported by `pan`.

Property	Config	Result	Depth	States
`pcb_mutex`	`MAX_INSTR 3`	violated	901	65 957
`no_self_wait`	`MAX_INSTR 3`	violated	379	188
`waiting_consistent`	`MAX_INSTR 3`	violated	962	69 699
`no_dup_ready`	`MAX_INSTR 2`	holds	n/a	20 861 451
`all_terminate`	`MAX_INSTR 3`, no `-f`	violated	14	25
`all_terminate`	`MAX_INSTR 3`, `-f`	violated	1069	493
`ready_dispatched`	`MAX_INSTR 3`, no `-f`	violated	14	25
`ready_dispatched`	`MAX_INSTR 3`, `-f`	violated	1088	11 372 991
deadlock	`MAX_INSTR 3, -DNOCLAIM`	violated	504	67 872

Six of the seven checks failed, but they do not describe six problems. They collapse into two defects.

The double dispatch race

`pcb_mutex` fails at depth 901 when `executing[]` reaches 2. After checking the counterexample against `manager.c`, the interleaving is as follows. Worker A dequeues process p and runs an instruction requesting a resource held by another process. `request_resource` then sets p to `WAITING` and adds it to `waitingq`. A releases `i_lock` and is about to check `running_p->state == WAITING` to decide whether to leave the loop. Before this happens, worker B releases the resource, causing `release_resource` to move p back to `readyq` and set its state to `READY`. B then dequeues p and starts running it. A reads the updated `READY` state, does not leave the loop, and continues running p as well.

`waiting_consistent` fails 61 steps later in the same part of the execution. The trail shows a PCB in the `WAITING` state even though the resource it was waiting for has already been released by the other worker. Since `release_resource` only wakes a waiting process when the resource matches the one it is waiting for, this PCB remains in `waitingq` with no way to return to the ready queue. It also continues holding a resource that it will never release.

The deadlock run is the same defect from the consequence side. Its final state has `instr_count[2]` at 4 with `MAX_INSTR` at 3, leaving a worker stuck at the selection on `instr_count` because neither guard is executable. In `manager.c` that is two threads both running `running_p->next_instruction = running_p->next_instruction->next` for the same PCB, so the second advances a pointer already at

the end of the list and dereferences past it. C defines no behaviour there, and a Promela control point with no executable transition is a state with no defined successor, so the abstraction has nothing defined exactly where the implementation has nothing defined.

The fix is to hold a lock across the state test and the branch that follows it, not only across `execute_instr`. Doing that with `i_lock` costs parallelism, since `i_lock` already serialises every instruction in the system; a per-PCB lock is the cheaper version, because then only threads touching the same PCB serialise.

Self deadlock in `request_resource`

This counterexample is much smaller than the others, failing at depth 379 after 188 states. The trail only contains one worker, and `executing[]` never becomes greater than 1, so there is no thread interleaving involved. This is simply a logic error that a single thread can reach on its own. The fix is to add a check before the availability test so that `resource->allocated == cur_pcb` is treated as a successful request. The same `!resource_found` branch is also used when the requested resource does not exist. These are two different cases, so the fix should distinguish between a resource already owned by the process, a resource held by another process, and a resource that does not exist.

This defect also explains both liveness failures and has another consequence that is not checked by the model. `detect_deadlock` searches for cycles in the waiting queue, and a process waiting for a resource that it already owns creates a cycle of length 1. The program would therefore report a deadlock even though it is not the type of deadlock the diagnostic is intended to detect. A single missing check therefore causes a safety violation, 2 liveness violations, and an incorrect deadlock diagnosis.

Liveness and fairness

Both liveness properties also fail without `-f`, at depth 14 after 25 states, and neither of those results is evidence of anything. The cycle they report is one in which `init` evaluates its loop guard and is never scheduled again, so `ready_q` is never seeded and a worker spins on an empty queue forever. No OpenMP runtime passes over a thread that can make progress indefinitely [7], and weak fairness excludes exactly those cycles [5], so every liveness result reported here uses `-f`. Fairness is not free: `ready_dispatched` finishes in 25 states without it and explores 11 372 991 with it.

The fair `all_terminate` counterexample ends with all three processes `WAITING`, process 1 holding resource 1 while waiting for resource 1, and both workers already exited because `check_terminate` counts every process as blocked. `ready_dispatched` fails for the same reason but exposes a separate design consequence; which is the stopping condition retires the workers as soon as everything is blocked, so a process can still be on the ready path when the system decides it is finished.

The property that holds

`no_dup_ready` was checked exhaustively at `MAX_INSTR 2` with 0 errors over 20 861 451 states in about 21 seconds, using 2 421 MB. The run also reported 0 of 12 states unreached in `init` and 0 of 119 unreached in `Worker`, which is what makes the pass worth quoting: every control point was visited, so the result is not an artefact of dead code in the abstraction [6].

The reduced bound is necessary because of the size of the state space. Failed properties are relatively cheap to check because the search stops when the first counterexample is found. However, a property that passes must explore all reachable states. The state space grows by about a factor of 50 for each additional instruction. 420,705 states at `MAX_INSTR 1` and 20 861 451 at 2, which would result in roughly 10^9 states at 3. With `pan` reporting 96 bytes per state, this would require around 100 GB of memory, compared to the 7 GB available.

The bound of 2 is reasonable because the shortest possible duplicate requires a PCB to be added to the queue twice before it is removed once. This requires one blocking request followed by a wakeup, so two instructions are enough to produce the smallest possible counterexample. Increasing the bound would therefore mainly allow longer versions of the same behaviour. This argument is based on the shortest witness rather than a formal proof.

It is also important to note that the model uses a channel for the ready queue. A channel cannot contain two references to the same PCB in the way that an incorrectly maintained linked list could; this result checks the enqueue and dequeue behaviour rather than the pointer manipulation performed by `enqueue_pcb`.

2 Program 2: MPI Master/Worker Othello Engine

2.1 The Promela Model

The model abstracts the MPI master and worker move evaluation in `my_player.c`: `run_master`, `execute_master`, `send_init_moves`, `send_next_move`, `pop_move` and `run_worker`. The referee is modelled as an environment process, `Referee`, because a Promela model must be closed and the communication flow through `comms.c` is fixed by the framework rather than by us. The board, the search recursion, `evaluate` and the Othello rules are all abstracted away. What survives of `minimax_pruning` is only which of three outcomes a search produces, which is the least the properties in Section 2.2 need.

Three constants control the size of the model, each using a `#ifndef` guard so that it can be set from the command line. `NUM_WORKERS` is set to 3 because `common/configs.py` in the Othello project uses `"threads": 4`. Since `comm_sz` is 4, this gives one master and three workers, matching the actual system.

`MAX_MOVES` must be greater than `NUM_WORKERS`. Otherwise, `send_init_moves` can empty the move stack before any worker returns a result, meaning the dynamic work redistribution branch is never reached. This was discovered using `pan`'s unreached statement report with `NUM_WORKERS 3, MAX_MOVES 3`, which showed that the redistribution send was unreachable. `MAX_MOVES 4` is therefore the smallest suitable value. Finally, `MAX_ROUNDS` limits the number of rounds in the `Referee` process so that each run terminates. It is set to 2 by default and varied between runs in Section 2.3.

Table 5 sets out thef correspondence between the implementation and the model.

Table 5: Mapping between the implementation and the model.

`my_player.c`	**Model**	**Reason**
`MPI_Bcast(&my_colour)`, `MPI_Bcast(board)`	`bcast_colour[]`, `bcast_board[]`	For each collective there is 1 channel per worker. The board carries no information any property needs, so it is reduced to the round number.
`MPI_Send`/`MPI_Recv` with `MOVE_TAG`, `NO_WORK_TAG`	`to_worker[]`, one buffered channel per worker	Direct: the tag distinguishes work from termination exactly as in the implementation.
`MPI_Iprobe(MPI_ANY_SOURCE)` + `MPI_Recv`	`to_master`, one shared channel	`MPI_ANY_SOURCE` does not distinguish senders, so a single channel every worker writes into is a direct abstraction, not a simplification.
`move_stack` (linked list)	`moves_left` counter	Nothing else reads the stack and no other process touches it, so only its size can affect any interleaving.
`minimax_pruning`	`evaluate_move` inline	The board, recursion and `evaluate` are removed. What survives is which of three outcomes a call produces, discussed below.
`run_master`'s if-chain over `comms.h` message types	`Master`'s `if` over `mtype`	Direct: the same branches are handled, and the same two are not.
The Ingenious Framework referee (reached over a socket via `comms.c`)	`Referee`, an environment process	Promela models must be closed [4]; `budget` bounds the messages so every run terminates.

The headline abstraction is `evaluate_move` (stands for the call a worker makes into `minimax_pruning`) and through it, the minimising branch of `minimax`. That branch initialises an accumulator to `INT_MAX` and loops over the legal moves at that node:

Listing 2: The minimising branch of `minimax`

```
minEval = INT_MAX;
```

```
legal_moves(moves, &number_of_moves, my_colour);
for (int i = 0; i < number_of_moves; i++)
{
  ...
  minEval = minimum(minEval, eval);
  beta = minimum(beta, eval);
  if (beta <= *alpha)
  {
    break;
  }
}
return minEval;
```

Three outcomes fall out of this; namely the loop never runs because there are no legal moves at that node, so the function returns its initial value unchanged; the loop runs once and stops, because the pruning test sits at the bottom rather than the top, so a poisoned or reduced-depth call returns its first child's value rather than a true minimum; or the loop runs to completion and returns a genuine evaluation. The model reproduces exactly these three outcomes as 3 branches of 1 `if`, resolved non-deterministically wherever more than one applies:

Listing 3: `evaluate_move`, the abstraction of `minimax_pruning`

```
inline evaluate_move(m,in_alpha,in_depth,out_score,out_alpha)
{
  if
  :: out_score = NO_EVAL
  :: (in_alpha == NO_EVAL || in_depth == DEPTH_REDUCED) ->
    select(out_score : 1 .. SCORE_MAX)
  :: (in_alpha != NO_EVAL && in_depth == DEPTH_FULL) ->
    out_score = true_score[m]
  fi;
  if
  :: out_score > in_alpha -> out_alpha = out_score
  :: out_score <= in_alpha -> out_alpha = in_alpha
  fi
}
```

`NO_EVAL` represents `INT_MAX` and is set to 9, which is larger than any valid score in the abstract domain `1..SCORE_MAX`. This gives it the same behaviour as `INT_MAX` for the comparisons made by the master. `true_score[]` is only used for verification and has no equivalent in the implementation. At the start of each round, it assigns a non-deterministic valid score to each move. This allows the model to compare the move selected by the master with the move that should have been selected. The actual implementation does not have such an array because the correct value of a move is only known from a complete search.

Two omissions in Table 6 are not free, and both are central to Section 2.3.

Table 6: What the model leaves out, and what each omission costs.

Omitted	**Cost**
Board contents, geometry, `is_stable`, board weights	None → No property depends on where a piece sits, only on the score a search returns for it.
Recursion inside `minimax` below the top call	None → It is thread-local and cannot affect any other process's interleaving.
The `num_moves`-to-depth mapping (11, 12, 13 or 14)	None → Nothing in the protocol depends on the specific depth, only on whether a search was cut short, so all four collapse to `DEPTH_FULL` and `DEPTH_REDUCED`.
The `time()` cutoff clock	Small → A free nondeterministic choice can fire the cutoff at any point in a round, which over-approximates a real clock rather than modelling one; a counter would multiply the state space for no benefit to the properties checked.
The `MPI_Iprobe` busy-wait, replaced by a blocking receive	Not free → This changes what an unanswered result looks like to Spin, discussed in Section 2.1.

Limits on interpretation

The limits in Section 1.1 apply here too, with the bound now set by the number of workers, moves and rounds.There is also a limitation specific to this program. The implementation uses the `MPI_Iprobe` busy-wait in `execute_master`, and the model replaces it with a blocking channel receive. If an `MPI_Iprobe` loop never receives a message, the implementation runs indefinitely, which Spin would detect as a non-progress cycle using `-l`; in the model, a blocking receive that never completes is an invalid end state, caught by the default safety check. In both cases the master never returns, so the abstraction represents an implementation livelock as a model deadlock. The results in Section 2.3 are read with this difference in mind.

2.2 Correctness Properties

Three variables have no counterpart in `my_player.c` and exist only so a property can be stated. This is the same role `executing[]` and `in_ready[]` play in the scheduler model. `round_done` is false while a round is in progress and true once the collection loop has finished and `best_move` is final. `best_true` is the largest `true_score` among the round's moves. `workers_done` counts how many workers have left their loop and can reach `MPI_Finalize`.

Five checks were run: four properties (as named `ltl` blocks), plus a deadlock check that is not an LTL property. Table 7 gives each formula as written in `model.pml`.

Table 7: The 4 LTL properties as written in `model.pml`.

Property	Class	Formula
`no_eval_chosen`	safety	`[] !(round_done && best_score == NO_EVAL)`
`best_is_maximal`	safety	`[] ((round_done && num_moves > 0) -> true_score[best_move] == best_true)`
`no_lost_results`	safety	`[] (round_done -> len(to_master) == 0)`
`workers_finish`	liveness	`<> (workers_done == NUM_WORKERS)`

`no_eval_chosen` targets the leaf case in the minimising branch of `minimax`. There is no guard for `number_of_moves == 0`, so a node with no legal moves returns `minEval` with its initial value of `INT_MAX`. This value is then used by `execute_master` in the comparison `if ((process_score > max_score) || (max_move == -1))`. Since `INT_MAX` is larger than every valid evaluation, the move that was never searched can be selected; this affects the correctness of the move chosen by the engine, rather than just its internal state.

`best_is_maximal` checks whether the move selected by `execute_master` is actually one of the best available moves. The condition `num_moves > 0` excludes the pass case, where `best_move` is 0 and `true_score[0]` has no meaning. Two issues can cause this property to fail. The first is the `NO_EVAL` problem described above. The second is that the master compares scores calculated under different conditions. The search depth can starts at 11, or 12 to 14 when there are few legal moves, and drops to 4 when the turn clock passes `cut_off`, while the shared `alpha` can cause some workers to prune earlier than others. Since the pruning check `if (beta <= *alpha) break;` is at the bottom of the loop, a depth-reduced or high-alpha search still evaluates its first child and can return a normal-looking score instead of `INT_MAX` (this makes the second issue harder to detect than the first). The fault injection results separating these two causes are given in Section 2.3.

`no_lost_results` is the property the whole work-sharing protocol rests on, and the one expected to hold. `to_master` stands for the `MPI_Iprobe` plus `MPI_Recv(MPI_ANY_SOURCE)` pair, and a single shared channel is the right abstraction because `MPI_ANY_SOURCE` does not differentiate senders. The `Iprobe` loop has no escape: the time cutoff lowers `depth` but never breaks it, so if the send and receive counts were ever unbalanced the master would spin forever. This property confirms the balance is exact.

`workers_finish` checks whether every worker eventually leaves its loop and reaches `MPI_Finalize`. A worker only exits when it receives a negative broadcast colour, and only the `GAME_TERMINATION` and `UNKNOWN` branches of `run_master` send this broadcast. However, `receive_message` in `comms.c` can also return `RECV_FAILED` or `CLIENT_DISCONNECTED`, and `run_master` has no branch for either case. Since `comms.h` was fixed by the project specification and couldn't be changed, handling these values is the

responsibility of the player. This failure occurs both with and without `-f`, confirming that it is caused by the missing branches rather than scheduling.

Deadlock is checked in the same way as Program 1, using a claim-free verifier compiled with `-DNOCLAIM`. It is useful to check alongside `workers_finish` because they identify the same problem in different ways: one shows that the workers never finish, while the other shows that the system has become stuck.

Two things were considered and not verified. A property comparing the round number a worker receives against `round_no` was drafted to check that the two broadcasts, `bcast_colour` and `bcast_board`, stay matched. Alpha monotonicity within a round, that `master_alpha` never decreases, was also considered and dropped. It is true by construction, since `alpha` is only ever raised by a comparison, so a property asserting it would establish nothing.

2.3 Verification Results and Analysis

Each run selects a claim using `./pan -a -N name`, and the deadlock check uses a second binary compiled with `-DNOCLAIM` from the same generated `pan.c`, as in Program 1. `NUM_WORKERS`, `MAX_MOVES` and `MAX_ROUNDS` are supplied to `spin` on the command line, and both binaries are rebuilt after each `spin -a`.

Table 8: Verification results. Depths and state counts as reported by `pan`.

Property	Config	Result	Depth	States
`no_eval_chosen`	`W3, M4, R1`	violated	205	5 336
`best_is_maximal`	`W3, M4, R1`	violated	255	106 138
`workers_finish`	`W3, M4, R1`	violated	31	203
deadlock	`W3, M4, R1, -DNOCLAIM`	violated	20	205
`no_eval_chosen`	`W2, M3, R1`	violated	172	1 046
`best_is_maximal`	`W2, M3, R1`	violated	222	17 286
`workers_finish`	`W2, M3, R1`	violated	28	77
deadlock	`W2, M3, R1, -DNOCLAIM`	violated	17	79
`no_lost_results`	`W2, M3, R1`	holds	n/a	7 676 089
`no_lost_results`	`W3, M4, R1, -DBITSTATE`	0 errors, approximate	436	655 692 780

`W`, `M` and `R` abbreviate `NUM_WORKERS`, `MAX_MOVES` and `MAX_ROUNDS`. `W3, M4, R1` matches the deployed system; `W2, M3, R1` is the reduced configuration, small enough to search exhaustively, used wherever a property must hold rather than merely fail fast.

The `NO_EVAL` leak

`no_eval_chosen` fails at depth 205 when a single worker reaches a node with no legal moves. As with `no_self_wait` in Program 1, this is a simple logic error rather than a race condition. The problem is also not limited to the first round. With `MAX_ROUNDS 2`, the property fails again at depth 341 after 5,683 states, after the first round has already finished. The match logs in `othello/evidence/` also show that this occurs in practice. In match `20250513_115728`, 22 of 60 rounds ended with a score of 2147483647, and workers received an `alpha` value of that value 76 times during the same match.

Two independent causes of a suboptimal move

`best_is_maximal` fails at depth 255, saying the engine played a worse move than one it had already evaluated, the same shape as round 47 vs. round 24 in the match log, where move 47 scored 1328 and move 24 scored `INT_MAX` and was played anyway. Section 2.2 gives two mechanisms that can cause this on their own, the fault injection runs in Table 9 isolate them.

Removing either cause on its own still leaves the property violated; the property only holds when both are removed. Fixing only the `minimax` leak would therefore not be enough.

The missing message-type branch

`workers_finish` fails at depth 31, while the deadlock check finds an invalid end state at depth 20 in the same configuration. Both failures are caused by the missing `RECV_FAILED` and `CLIENT_DISCONNECTED` branches in `run_master`. The smallest counterexample does not require a move round. With `MAX_ROUNDS`

Table 9: Fault isolation on `best_is_maximal`, `W2, M3, R1`.

Flags	Result	Depth	States
none	violated	222	17 286
`-DNO_TIME_CUTOFF`	violated	218	10 026
`-DNO_LEAK`	violated	280	47 056
`-DNO_TIME_CUTOFF -DNO_LEAK`	holds	n/a	626 037

`0`, the deadlock still occurs at depth 20 after 205 states, while `workers_finish` fails at depth 31 after 203 states. This shows that the defect is entirely within the referee message loop and is unrelated to move evaluation.

The property that probably holds

`no_lost_results` was checked exhaustively with `W2, M3, R1`, finding 0 errors across 7 676 089 states in 11.4 seconds. All 4 proctypes also had 0 unreached statements. At the deployed configuration, `W3, M4, R1`, the exhaustive search was stopped by the kernel's out-of-memory killer. The state vector is 244 bytes, so 7GB can store roughly 2.6×10^7 states, and the search exceeded this before finishing. As a fallback, bitstate storage was used. This stores states in a hash-based bit array, meaning different states can map to the same entry and be treated as already visited. The run found 0 errors across 655 692 780 states in about 1,370 seconds. However, its hash factor of 6.55 is well below 100, which indicates that the search was not close to exhaustive. Therefore, the result only shows that no counterexample was found in the approximate search of roughly 6.6×10^8 states. The exhaustive two-worker result provides stronger evidence, so both results are reported.

The reduction to two workers is reasonabel for this property. A lost result requires the master to finish a round while a result is still waiting in the queue. The shortest example only needs one worker whose result is processed and another whose result is missed, which can occur with two workers. Adding a third worker would mainly allow longer versions of the same behaviour. This is an argument based on the shortest counterexample rather than a proof, so the result only applies within the verified bound.

One search-order detail is important because it affects the reported state counts without changing their meaning. The 4 terminal message branches in `proctype Referee` are placed before the `budget > 0` branch. Since depth first search explores the nondeterministic `do` branches in the order they appear, this allows a termination counterexample to be found after only a few hundred states, rather than after exploring a full move round.

Proposed fixes

Each of the three defects needs a different kind of fix.

- The `NO_EVAL` leak is the easiest to fix. The minimising branch of `minimax` needs a guard for `number_of_moves == 0` before the loop, since a node with no legal moves is a normal pass in Othello, not an error. It should return a real evaluation of the current position using `evaluate`, rather than the `INT_MAX` value used to initialise the accumulator. The maximising branch has the same issue with `INT_MIN`, so both branches should include the guard even though only the minimising case was found here.

- The comparability defect requires a design change rather than just an extra test. The master lowers `depth` from 11-4 during a round, then compares scores from both depths as if they were on the same scale. The simplest fix is to apply the new depth from the next round, so that all scores in a round use the same search depth.

- The missing message branch is the smallest fix of the three. Rather than adding 2 more `else if` arms for `RECV_FAILED` and `CLIENT_DISCONNECTED`, the safer form is to give the if-chain a final `else` that sets `running = 0` and issues the termination broadcast, since that also covers any value `comms.h` might gain later. Either way the workers stop blocking in `MPI_Bcast` and reach `MPI_Finalize`.

Table 10: Environment for every run reported in this document.

Model checker	Spin Version 6.5.2, 6 December 2019
Compiler	gcc
Platform	Ubuntu Linux
Memory	7 GB RAM, 1 GB swap

A Reproducing the verification results

Table 10 gives the environment. iSpin was not used for any result in either program, and all runs were performed from the command line.

Program 1: FCFS scheduler

Model version: commit `517e9be`.

Listing 4: Build

```
cd Modelchecking/scheduler
spin -a model.pml
gcc -O2 -o pan pan.c
gcc -O2 -DNOCLAIM -o pan_noclaim pan.c
```

Listing 5: Verification runs, default configuration (MAX_INSTR 3)

```
./pan -a -N pcb_mutex
./pan -a -N no_self_wait
./pan -a -N waiting_consistent
./pan -a -N all_terminate -m50000
./pan -a -N all_terminate -f -m50000
./pan -a -N ready_dispatched -m50000
./pan -a -N ready_dispatched -f -m50000 -w26
./pan_noclaim
```

Listing 6: Reduced configuration (MAX_INSTR 2), for the property that holds

```
spin -a -DMAX_INSTR=2 model.pml
gcc -O2 -o pan pan.c
./pan -a -N no_dup_ready -m20000 -w26
```

Program 2: Othello

Model version: commit `70b73d2`.

Every result in Table 8 and Table 9, except the three-worker `no_lost_results` run below, is produced by a single script:

Listing 7: Build and run everything

```
cd Modelchecking/othello
./verify.sh
```

This regenerates `pan.c` and rebuilds both binaries before each group of runs, writes every error trail to `results/`, the full `pan` output of every run to `results/logs/`, and a summary table to `results/summary.txt`. Every run is given a wall clock limit of one hour. Passing `quick` as the first argument skips the two `no_lost_results` runs: the three-worker one, which exhausts memory, and the two-worker one, which at 11.4 seconds is the longest run that completes.

The three-worker `no_lost_results` run is attempted by the script but is not expected to finish. It is the run the out-of-memory killer stops, and it appears in `summary.txt` as `INCOMPLETE` with coverage `unknown` rather than as a result. Its bitstate fallback is not part of the script, because it takes roughly 23 minutes and is a deliberate one-off rather than a routine result, so it is run manually:

Listing 8: Three-worker `no_lost_results`, bitstate fallback

```
spin -DNUM_WORKERS=3 -DMAX_MOVES=4 -DMAX_ROUNDS=1 -a model.pml
gcc -O2 -DBITSTATE -o pan_bit pan.c
./pan_bit -a -N no_lost_results -m100000 -w32
```

AI Declaration

Generative AI was used in an assisting role throughout the project.

For the code, it was used to provide guidance when encountering difficulties and assist in identifying the cause of issues.

For the report, generative AI was used to provide guidance on the overall structure, improve grammar and sentence construction, and suggest clearer wording for technical explanations. The final content, technical decisions, analysis, results, and conclusions were my own work.

References

[1] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, Cambridge, MA, 2008.

[2] Gerard J. Holzmann. Promela reference: `atomic`(3). `https://spinroot.com/spin/Man/atomic.html`, 2025. Accessed Aug 2026.

[3] Gerard J. Holzmann. Pan verification options. `https://spinroot.com/spin/Man/Pan.html`, 2025. Accessed Aug 2026.

[4] Cornelia P. Inggs. Promela. Lecture slides, CS796 Software Verification and Spin Model Checking, Stellenbosch University, 2025. Source: The Spin Model Checker, G. J. Holzmann.

[5] Cornelia P. Inggs. Properties. Lecture slides, CS796 Software Verification and Spin Model Checking, Stellenbosch University, 2025. Source: The Spin Model Checker, G. J. Holzmann.

[6] Cornelia P. Inggs. Spin. Lecture slides, CS796 Software Verification and Spin Model Checking, Stellenbosch University, 2025. Source: The Spin Model Checker, G. J. Holzmann.

[7] OpenMP Architecture Review Board. OpenMP application programming interface, version 5.2. `https://www.openmp.org/specifications/`, 2021.